

semmcci: Monte Carlo Confidence Intervals

Ivan Jacob Agaloos Pesigan

Installation

You can install the CRAN release of `semmcci` with:

```
install.packages("semmcci")
```

You can install the development version of `semmcci` from [GitHub](#) with:

```
if (!require("pak")) install.packages("pak")  
pak::pkg_install("jeksterslab/semmcci")
```

Documentation

See [GitHub Pages](#) for package documentation.

Description

In the Monte Carlo method, a sampling distribution of parameter estimates is generated from the multivariate normal distribution using the parameter estimates and the sampling variance-covariance matrix. Confidence intervals for defined parameters are generated by obtaining percentiles corresponding to $100(1 - \alpha)\%$ from the generated sampling distribution, where α is the significance level.

Monte Carlo confidence intervals for free and defined parameters in models fitted in the structural equation modeling package `lavaan` can be generated using the `semmcci` package. The package has two main functions, namely, `MC()` and `MCStd()`. The output of `lavaan` is passed as the first argument to the `MC()` function to generate Monte Carlo confidence intervals. Monte Carlo confidence intervals for the standardized estimates can also be generated by passing the output of the `MC()` function to the `MCStd()` function. A description of the package and code examples are presented in Pesigan and Cheung (2024).

Example

A common application of the Monte Carlo method is to generate confidence intervals for the indirect effect. In the simple mediation model, variable **X** has an effect on variable **Y**, through a mediating variable **M**. This mediating or indirect effect is a product of path coefficients from the fitted model.

```
library(semmcci)
library(lavaan)
```

Data

```
n <- 1000
X <- rnorm(n = n)
M <- 0.50 * X + rnorm(n = n)
Y <- 0.25 * X + 0.50 * M + rnorm(n = n)
data <- data.frame(X, M, Y)
```

Model Specification

The indirect effect is defined by the product of the slopes of paths **X** to **M** labeled as **a** and **M** to **Y** labeled as **b**. In this example, we are interested in the confidence intervals of `indirect` defined as

the product of **a** and **b** using the `:=` operator in the **lavaan** model syntax.

```
model <- "  
  Y ~ cp * X + b * M  
  M ~ a * X  
  indirect := a * b  
  direct := cp  
  total := cp + (a * b)  
"
```

Model Fitting

We can now fit the model using the `sem()` function from **lavaan**.

```
fit <- sem(data = data, model = model)
```

Monte Carlo Confidence Intervals

The `fit` **lavaan** object can then be passed to the `MC()` function to generate Monte Carlo confidence intervals.

```
MC(fit, R = 20000L, alpha = 0.05)  
  
#> Monte Carlo Confidence Intervals  
#>      est      se      R  2.5% 97.5%  
#> cp      0.2460 0.0368 20000 0.1737 0.3186  
#> b      0.5044 0.0323 20000 0.4409 0.5669  
#> a      0.5654 0.0314 20000 0.5036 0.6264  
#> Y~~Y    1.0357 0.0461 20000 0.9441 1.1266  
#> M~~M    0.9901 0.0443 20000 0.9041 1.0783
```

```
#> X~~X      1.0107 0.0000 20000 1.0107 1.0107
#> indirect 0.2852 0.0242 20000 0.2392 0.3336
#> direct    0.2460 0.0368 20000 0.1737 0.3186
#> total     0.5312 0.0357 20000 0.4606 0.6013
```

Standardized Monte Carlo Confidence Intervals

Standardized Monte Carlo Confidence intervals can be generated by passing the result of the `MC()` function to `MCStd()`.

Note: We recommend setting `fixed.x = FALSE` when generating standardized estimates and confidence intervals to model the variances and covariances of the predictors if they are assumed to be random.

```
fit <- sem(data = data, model = model, fixed.x = FALSE)
unstd <- MC(fit, R = 20000L, alpha = 0.05)
vcov(unstd)
```

#>	cp	b	a	Y~~Y	M~~M
#> cp	1.371762e-03	-6.019668e-04	-6.490438e-07	3.691900e-06	-5.473147e-06
#> b	-6.019668e-04	1.047307e-03	7.419904e-06	8.452616e-06	1.144704e-05
#> a	-6.490438e-07	7.419904e-06	9.720496e-04	4.447635e-06	-5.869640e-06
#> Y~~Y	3.691900e-06	8.452616e-06	4.447635e-06	2.174852e-03	3.380535e-06
#> M~~M	-5.473147e-06	1.144704e-05	-5.869640e-06	3.380535e-06	1.935031e-03
#> X~~X	1.462816e-05	-1.316235e-05	1.566587e-05	9.335139e-06	-1.911471e-05
#> indirect	-3.404011e-04	5.963409e-04	4.942102e-04	6.799185e-06	4.148095e-06
#> direct	1.371762e-03	-6.019668e-04	-6.490438e-07	3.691900e-06	-5.473147e-06
#> total	1.031361e-03	-5.625844e-06	4.935612e-04	1.049109e-05	-1.325052e-06
#>	X~~X	indirect	direct	total	
#> cp	1.462816e-05	-3.404011e-04	1.371762e-03	1.031361e-03	

```
#> b      -1.316235e-05  5.963409e-04 -6.019668e-04 -5.625844e-06
#> a      1.566587e-05  4.942102e-04 -6.490438e-07  4.935612e-04
#> Y~~Y    9.335139e-06  6.799185e-06  3.691900e-06  1.049109e-05
#> M~~M   -1.911471e-05  4.148095e-06 -5.473147e-06 -1.325052e-06
#> X~~X    2.058503e-03  4.703091e-07  1.462816e-05  1.509847e-05
#> indirect 4.703091e-07  5.875727e-04 -3.404011e-04  2.471716e-04
#> direct  1.462816e-05 -3.404011e-04  1.371762e-03  1.031361e-03
#> total   1.509847e-05  2.471716e-04  1.031361e-03  1.278533e-03
```

MCStd(unstd)

```
#> Standardized Monte Carlo Confidence Intervals
#>      est      se      R  0.05%   0.5%   2.5%  97.5%  99.5%  99.95%
#> cp      0.1972 0.0294 20000 0.1026 0.1223 0.1392 0.2547 0.2745 0.2947
#> b      0.4609 0.0273 20000 0.3702 0.3897 0.4072 0.5135 0.5293 0.5525
#> a      0.4960 0.0239 20000 0.4122 0.4327 0.4484 0.5424 0.5560 0.5709
#> Y~~Y    0.6585 0.0243 20000 0.5731 0.5932 0.6092 0.7049 0.7173 0.7309
#> M~~M    0.7540 0.0237 20000 0.6740 0.6908 0.7058 0.7989 0.8128 0.8301
#> X~~X    1.0000 0.0000 20000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
#> indirect 0.2286 0.0178 20000 0.1724 0.1838 0.1939 0.2636 0.2763 0.2896
#> direct  0.1972 0.0294 20000 0.1026 0.1223 0.1392 0.2547 0.2745 0.2947
#> total   0.4259 0.0260 20000 0.3368 0.3583 0.3742 0.4761 0.4911 0.5080
```

References

Pesigan, I. J. A., & Cheung, S. F. (2024). Monte Carlo confidence intervals for the indirect effect with missing data. *Behavior Research Methods*, 56(3), 1678–1696. <https://doi.org/10.3758/s13428-023-02114-4>

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://www.R-project.org/>